

ADR-001-llm-provider-interface

ADR-001: LLM Provider Interface Design

Status

Accepted

Date

2026-01-08

Context

HelixCode is an enterprise-grade distributed AI development platform that needs to integrate with multiple Large Language Model (LLM) providers. The platform must support:

1. **Local providers** for privacy-sensitive deployments and development environments:
 - Ollama
 - Llama.cpp
 - vLLM
 - LocalAI
 - KoboldAI
2. **Cloud providers** for production scalability and access to state-of-the-art models:
 - OpenAI (GPT-4, GPT-4 Turbo, o1)
 - Anthropic (Claude 3, Claude 3.5)
 - Google Gemini (Gemini Pro, Gemini Ultra)
 - xAI (Grok)
 - AWS Bedrock (multi-model)
 - Azure OpenAI
 - Groq
 - OpenRouter
 - Qwen
 - Vertex AI
3. **Operational requirements:**
 - Automatic failover when a provider becomes unavailable
 - Load balancing across multiple providers
 - Cost optimization through provider selection strategies
 - Health monitoring and latency tracking
 - Streaming response support for real-time feedback
 - Model capability detection and matching

The challenge was to create a unified interface that abstracts away provider-specific details while maintaining the ability to leverage provider-specific features when needed.

Decision

We implemented a unified `Provider` interface that all LLM providers must implement, combined with a provider registry, health monitoring, and intelligent selection strategies.

Core Interface

```
type Provider interface {
    // Core generation methods
    Generate(ctx context.Context, request *LLMRequest)
        (*LLMResponse, error)
    GenerateStream(ctx context.Context, request *LLMRequest, ch
        chan<- LLMResponse) error

    // Provider metadata
    GetType() ProviderType
    GetName() string
    GetModels() []ModelInfo
    GetCapabilities() []ModelCapability

    // Health and lifecycle
    IsAvailable(ctx context.Context) bool
    GetHealth(ctx context.Context) (*ProviderHealth, error)
    Close() error
}
```

Request/Response Model

The `LLMRequest` structure provides a provider-agnostic way to specify generation parameters:

```
type LLMRequest struct {
    ID          uuid.UUID
    Model        string
    Messages    []Message
    MaxTokens    int
    Temperature float64
    TopP         float64
    Stream       bool
    Tools        []Tool
    // Provider-specific options
    Options      map[string]interface{}
}
```

Provider Selection Strategies

Multiple selection strategies are supported:

1. **Performance-based:** Selects provider with lowest latency
2. **Cost-based:** Selects cheapest provider for the request

3. **Availability-based:** Selects most reliable provider
4. **Round-robin:** Distributes load evenly
5. **Capability-based:** Selects provider that best matches required capabilities

Health Monitoring

Each provider includes health monitoring with: - Periodic health checks (configurable interval) - Latency tracking and averaging - Error count tracking - Automatic status transitions (healthy -> degraded -> unhealthy)

Automatic Fallback

When `llm.selection.fallback_enabled` is configured: 1. Primary provider is selected based on strategy 2. If primary fails, secondary provider is selected 3. Continues through provider list until success or exhaustion 4. Failed providers are temporarily excluded from selection

Consequences

Positive

1. **Vendor Flexibility:** Easy to switch between providers without code changes
2. **Cost Optimization:** Can route requests to cheaper providers for non-critical tasks
3. **High Availability:** Automatic failover ensures continuous operation
4. **Performance Optimization:** Load balancing prevents single-provider bottlenecks
5. **Future-Proofing:** New providers can be added by implementing the interface
6. **Testability:** Mock providers can be easily created for testing
7. **Local Development:** Can develop offline with local providers like Ollama

Negative

1. **Abstraction Overhead:** Some provider-specific features may be harder to access
2. **Configuration Complexity:** Multiple providers require careful configuration
3. **Debugging Complexity:** Issues may be harder to diagnose across providers
4. **Capability Variance:** Not all providers support all features (tools, vision, etc.)

Neutral

1. **Learning Curve:** Team needs to understand the abstraction layer
2. **Maintenance Burden:** Each new provider requires implementation and testing

Alternatives Considered

Alternative 1: Direct Provider Integration

Description: Directly integrate with each provider without abstraction.

Pros: - Full access to provider-specific features - No abstraction overhead - Simpler for single-provider deployments

Cons: - No automatic failover - Switching providers requires code changes - Testing requires actual provider access - Cannot leverage multiple providers simultaneously

Why Rejected: The multi-provider requirement and enterprise reliability needs made this approach insufficient.

Alternative 2: Third-Party Abstraction Layer (LiteLLM, LangChain)

Description: Use an existing abstraction library.

Pros: - Pre-built abstractions - Community support - Regular updates

Cons: - External dependency risk - Limited control over behavior - May not support all required providers - Performance overhead from additional layer - Difficulty customizing selection strategies

Why Rejected: The need for custom selection strategies, health monitoring, and tight integration with HelixCode's architecture favored a native implementation.

Alternative 3: Message Queue-Based Routing

Description: Use a message queue to route requests to provider-specific workers.

Pros: - Natural load balancing - Easy horizontal scaling - Decoupled architecture

Cons: - Additional infrastructure complexity - Higher latency for streaming - Operational overhead - More complex error handling

Why Rejected: The latency requirements for interactive development workflows made message queue overhead unacceptable.

Implementation Notes

- Provider implementations are in `internal/llm/` with naming convention `{provider}_provider.go`
- Health monitoring runs in background goroutines
- Provider registry maintains singleton instances
- Configuration is loaded from `config/config.yaml` with environment variable overrides
- Streaming uses channels for backpressure-aware delivery

Related Decisions

- ADR-004: Workflow Execution Model (uses LLM providers for code generation)
- ADR-003: Memory Provider Strategy (uses similar provider pattern)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/llm/local_provider.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/llm/anthropic_provider.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/llm/README.md`

- /run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/
internal/llm/LOCAL_PROVIDERS.md